Aula Prática

Operações Coletivas MPI com Python

Broadcast • Scatter • Gather • Reduce



Márcio Augusto E. de Moraes

Por que esta aula?

Antes dos desafios maiores, precisamos dominar o fluxo de dados entre processos.

- Os desafios usam comunicação coletiva em vez de vários send/recv manuais.
- Cada operação resolve um padrão de comunicação diferente.
- O foco aqui é entender para onde os dados vão e o que volta ao processo root.
- Depois disso, os alunos aplicam as operações em problemas numéricos.

Pergunta-guia

Qual operação MPI devo usar em cada etapa do algoritmo?

aula intermediária

pré-desafios

Objetivos de aprendizagem

Ao final, o aluno deve conseguir reconhecer e aplicar as quatro operações coletivas.

- Distinguir comunicação ponto a ponto e comunicação coletiva.
- Enviar uma mesma informação para todos os processos com broadcast.
- Distribuir blocos diferentes de dados com scatter.
- Reunir resultados individuais com gather.
- Combinar resultados parciais com reduce.
- Resolver um problema paralelo usando o padrão broadcast → scatter → processamento → reduce.

Relembrando MPI em Python

O mesmo programa é executado simultaneamente por vários processos.

```
from mpi4py import MPI

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

print(f"Processo {rank} de {size}")
```

```
mpiexec -n 4 python programa.py
```

- `rank` identifica o processo dentro do comunicador.
- `size` informa quantos processos participam da execução.
- `COMM_WORLD` é o comunicador com todos os processos.

Processo root

Em muitas operações coletivas, um processo coordena a distribuição ou recepção dos dados.

```
root = 0
```



- O root pode iniciar uma comunicação.
- Pode distribuir dados para os demais processos.
- Pode receber ou combinar resultados finais.

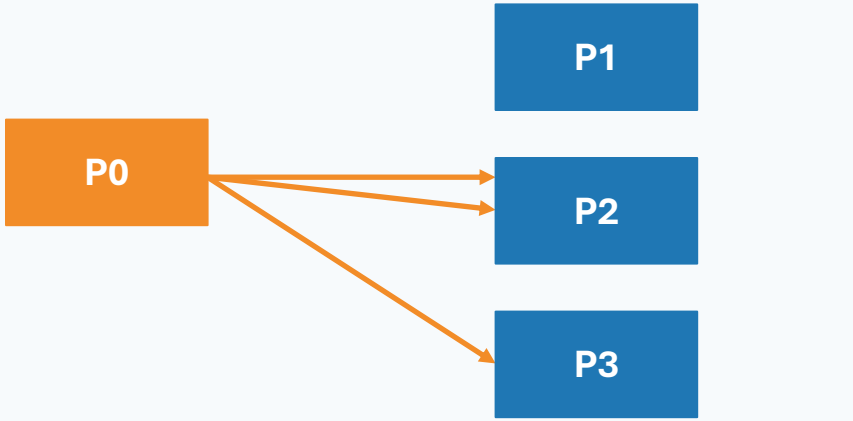
Ponto a ponto × coletiva

A comunicação coletiva envolve todos os processos do comunicador.

Ponto a ponto



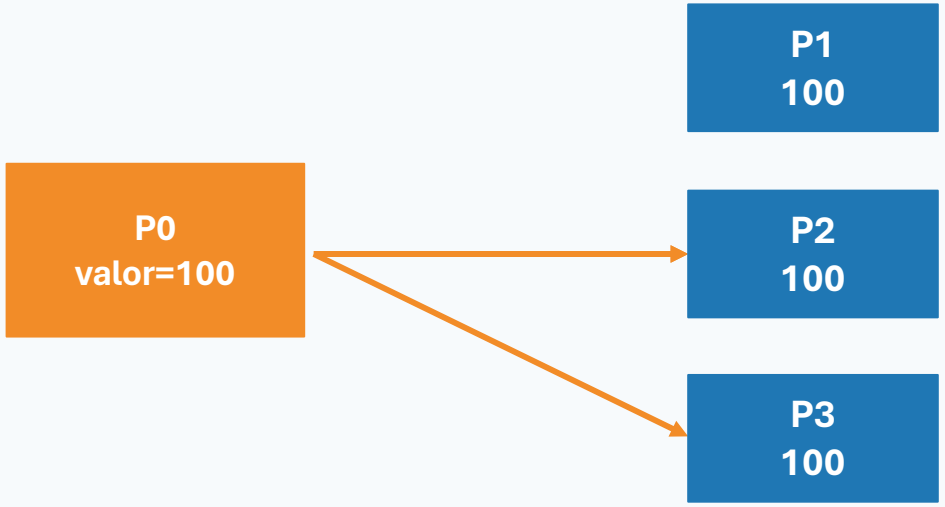
Coletiva



Regra: todos os processos do comunicador devem chamar a operação coletiva.

Broadcast: um envia para todos

Use quando todos os processos precisam conhecer a mesma informação.




```
numero = comm.bcast(numero, root=0)
```

- Parâmetros de simulação.
- Número de intervalos.
- Constantes físicas.
- Configuração do experimento.

Exemplo 1: Broadcast simples

Inicialmente apenas o processo 0 possui o valor. Depois, todos recebem o mesmo valor.

```python
from mpi4py import MPI

comm = MPI.COMM_WORLD
rank = comm.Get_rank()

if rank == 0:
    numero = 100
else:
    numero = None

numero = comm.bcast(numero, root=0)

print(f"Processo {rank}: numero = {numero}")
```

```
mpiexec -n 4 python exemplo_broadcast.py
```

Saída esperada

```
Processo 0: numero = 100
Processo 1: numero = 100
Processo 2: numero = 100
Processo 3: numero = 100
```

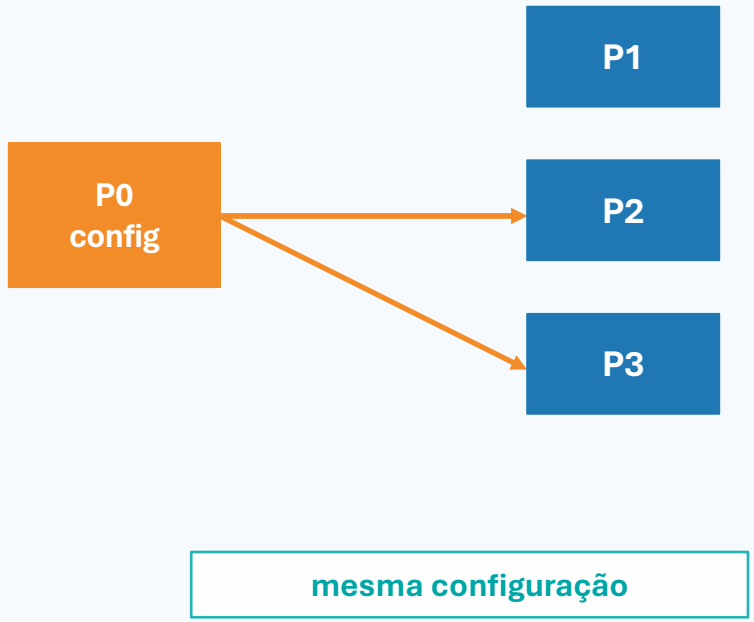
A ordem das linhas pode mudar.

Exemplo 2: Broadcast de parâmetros

O objeto enviado pode ser um número, texto, lista ou dicionário Python.

```
if rank == 0:
    parametros = {
        "N": 1000000,
        "inicio": 0,
        "fim": 100,
        "metodo": "Monte Carlo"
    }
else:
    parametros = None

parametros = comm.bcast(parametros, root=0)
```

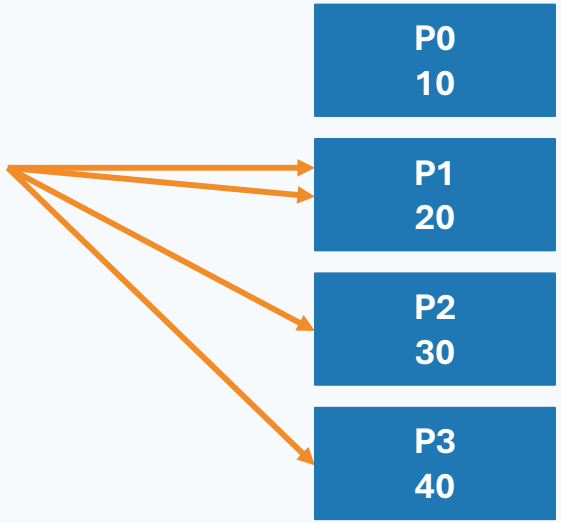


Scatter: distribuir partes diferentes

Use quando cada processo deve receber uma parte dos dados.

P0 possui:

```
[10, 20, 30, 40]
```




```
valor = comm.scatter(dados, root=0)
```

Broadcast: mesmo dado para todos
Scatter: dados diferentes

Exemplo 3: Scatter simples

A lista de entrada deve conter um item para cada processo.

```
from mpi4py import MPI

comm = MPI.COMM_WORLD
rank = comm.Get_rank()

if rank == 0:
    dados = [10, 20, 30, 40]
else:
    dados = None

valor = comm.scatter(dados, root=0)

print(f"Processo {rank} recebeu {valor}")
```

Com 4 processos:

```
P0 recebe 10
P1 recebe 20
P2 recebe 30
P3 recebe 40
```

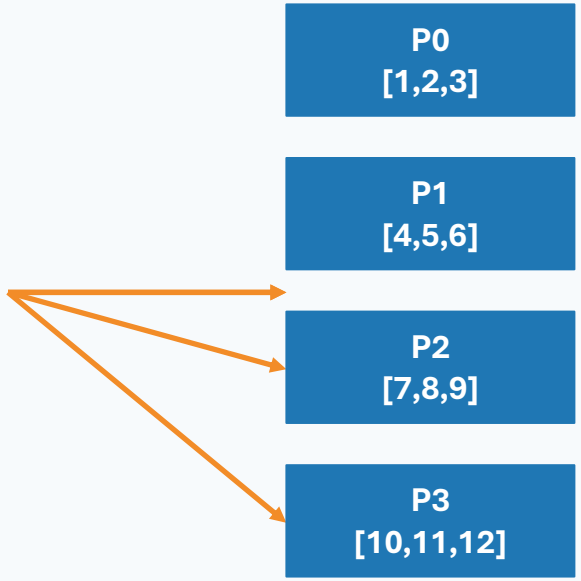
Se usar -n 4, a lista precisa ter 4 elementos.

Scatter de blocos

Em problemas reais, cada processo normalmente recebe um bloco de dados.

```
dados = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9],
    [10, 11, 12]
]

bloco = comm.scatter(dados, root=0)
```



decomposição dos dados

Depois do Scatter: processamento local

Cada processo trabalha apenas com sua própria parte do problema.

```
bloco = comm.scatter(dados, root=0)

soma_local = sum(bloco)

print(f"Processo {rank}: bloco={bloco},
soma={soma_local}")
```

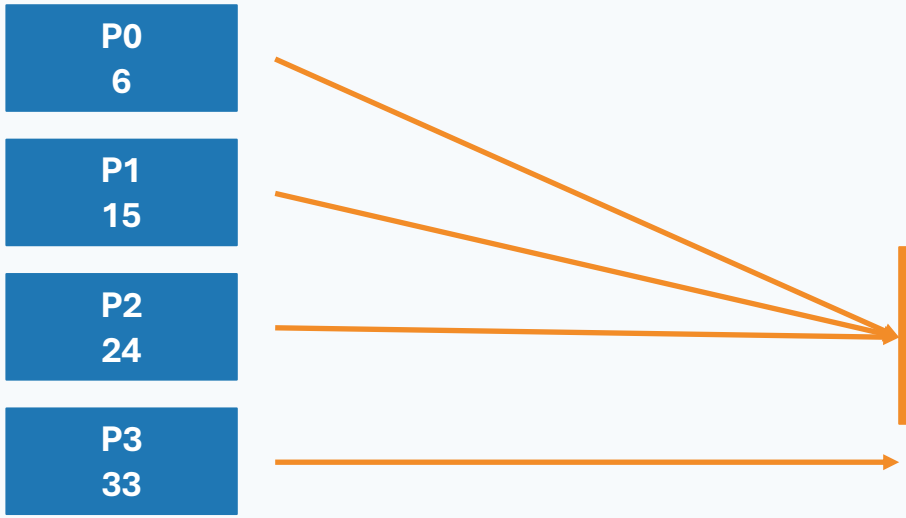
Resultados locais

```
P0 → 6
P1 → 15
P2 → 24
P3 → 33
```

Agora precisamos reunir ou combinar esses resultados.

Gather: reunir resultados individuais

Use quando o processo root precisa receber todos os resultados separadamente.






```
resultados =
comm.gather(soma_local, root=0)
```

Gather preserva os resultados individuais.

Exemplo 4: Gather

Cada processo calcula um valor local; o root recebe uma lista com todos os valores.

```
from mpi4py import MPI

comm = MPI.COMM_WORLD
rank = comm.Get_rank()

resultado_local = rank * 10

resultados = comm.gather(resultado_local, root=0)

if rank == 0:
    print("Resultados recebidos:", resultados)
```

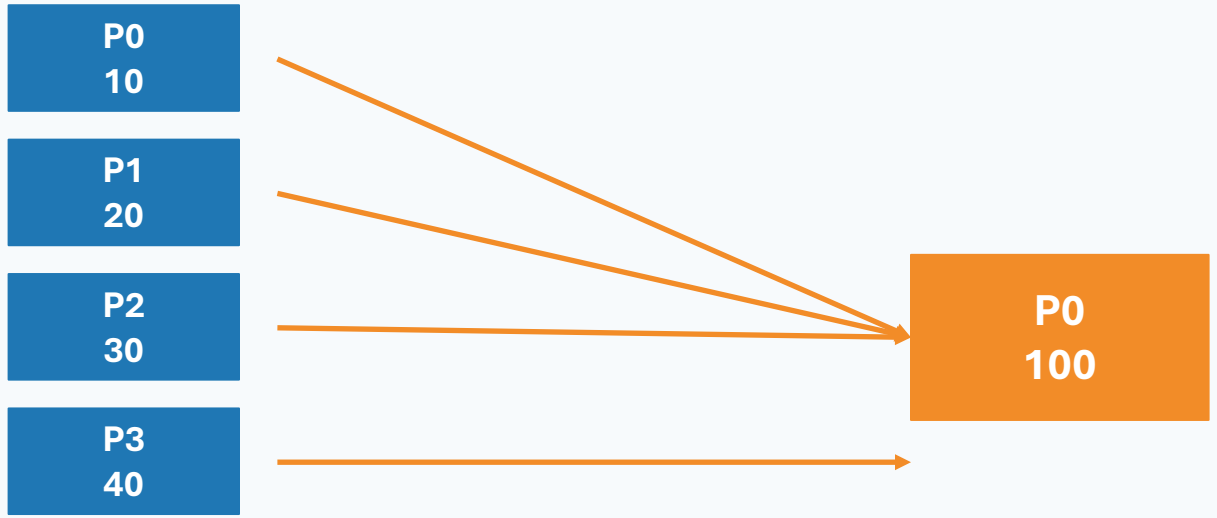
```
mpiexec -n 4 python exemplo_gather.py

Resultados recebidos: [0, 10, 20, 30]
```

Use gather quando a análise posterior precisa de cada valor separadamente.

Reduce: combinar resultados

Use quando não é necessário guardar todos os resultados locais, apenas um resultado global.




```
soma = comm.reduce(valor_local,
                   op=MPI.SUM,
                   root=0)
```

Resultado combinado no root

Resultados locais

Exemplo 5: Soma usando Reduce

A operação MPI.SUM soma automaticamente os valores locais.

```
from mpi4py import MPI

comm = MPI.COMM_WORLD
rank = comm.Get_rank()

valor_local = (rank + 1) * 10

soma = comm.reduce(valor_local, op=MPI.SUM, root=0)

if rank == 0:
    print("Soma global:", soma)
```

```
mpiexec -n 4 python exemplo_reduce.py

Soma global: 100
```

- MPI.SUM: soma
- MPI.MAX: maior valor
- MPI.MIN: menor valor
- MPI.PROD: produto

Exemplo 6: Maior valor global

A redução também pode combinar resultados com máximo, mínimo ou produto.

```python
from mpi4py import MPI

comm = MPI.COMM_WORLD
rank = comm.Get_rank()

valor = rank * 15 + 3
print(f"Processo {rank}: {valor}")

maior = comm.reduce(valor, op=MPI.MAX, root=0)

if rank == 0:
    print("Maior valor:", maior)
```

Com 4 processos:

```
P0 → 3
P1 → 18
P2 → 33
P3 → 48

Maior valor: 48
```

op=MPI.MAX

Gather ou Reduce?

A diferença depende do que você precisa obter no final.

Valores locais: 10, 20, 30, 40

Gather

```
[10, 20, 30, 40]
```

- Use quando precisa analisar cada resultado individual.

Reduce + SUM

```
100
```

- Use quando precisa apenas do resultado combinado.

Padrão 1: Scatter + processamento + Gather

Distribuir blocos, calcular resultados locais e reunir a lista final no root.




```
bloco = comm.scatter(dados, root=0)

soma_local = sum(bloco)

resultados = comm.gather(soma_local, root=0)
```

Bom quando o relatório final precisa de todos os resultados locais.

Padrão 2: Scatter + processamento + Reduce

Distribuir blocos, calcular resultados locais e combinar em um único resultado global.




```
bloco = comm.scatter(dados, root=0)

soma_local = sum(x*x for x in bloco)

soma_global = comm.reduce(soma_local, op=MPI.SUM, root=0)
```

Bom quando só precisamos da soma, máximo, mínimo ou outro resultado combinado.

Exemplo completo: soma dos quadrados

Calcular $S = \Sigma x_i^2$ usando scatter e reduce.

```
from mpi4py import MPI
import numpy as np

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

if rank == 0:
    dados = np.arange(1, 1001)
    blocos = np.array_split(dados, size)
else:
    blocos = None

bloco = comm.scatter(blocos, root=0)
soma_local = np.sum(bloco ** 2)

soma_global = comm.reduce(soma_local, op=MPI.SUM, root=0)

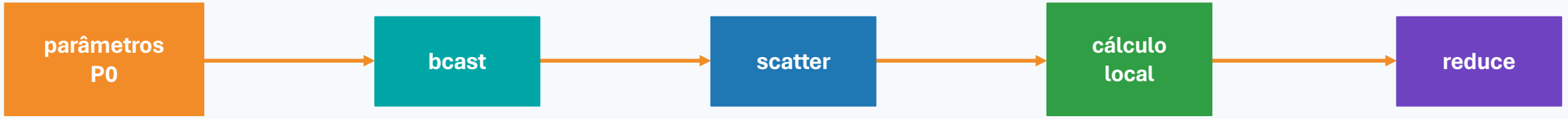
if rank == 0:
    print("Resultado:", soma_global)
```

- `np.array_split` ajusta automaticamente os blocos ao número de processos.
- Cada processo calcula sua soma local.
- `reduce` combina as somas locais em uma soma global.

```
mpiexec -n 2 python soma_quadrados.py
mpiexec -n 4 python soma_quadrados.py
mpiexec -n 8 python soma_quadrados.py
```

Padrão completo: Broadcast + Scatter + Reduce

Esse padrão aparece em muitos problemas de simulação e análise numérica.




```
N = comm.bcast(N, root=0)
expoente = comm.bcast(expoente, root=0)

bloco = comm.scatter(blocos, root=0)
resultado_local = np.sum(bloco ** expoente)

resultado_global = comm.reduce(resultado_local, op=MPI.SUM, root=0)
```

padrão fundamental

Quando usar cada operação?

Escolha a operação pelo movimento dos dados.

Operação	Movimento	Use quando...
`bcast()`	um → todos	todos precisam da mesma informação
`scatter()`	dividir	cada processo recebe uma parte diferente
`gather()`	reunir	root precisa dos resultados individuais
`reduce()`	combinar	root precisa de um resultado global

Regra prática: se o resultado final é uma soma, média, máximo ou mínimo, provavelmente reduce é melhor que gather.

Exercício 1: médias locais

Distribuir blocos de temperaturas e reunir as médias no processo 0.

```
temperaturas = [
    [20, 22, 21],
    [25, 27, 24],
    [18, 19, 20],
    [30, 32, 31]
]
```

- Use scatter para entregar um bloco a cada processo.
- Cada processo calcula sua média local.
- Use gather para reunir as médias no root.
- Resultado esperado: 21, 25.33, 19, 31.

scatter

gather

Exercício 2: soma de 1 até 1000

Distribuir o vetor e combinar as somas locais.

Calcular:

$$S = 1 + 2 + \ldots + 1000$$

```
S_teorico = N * (N + 1) / 2
```

- Crie um vetor com NumPy.
- Divida o vetor usando np.array_split.
- Use scatter para distribuir os blocos.
- Use reduce com MPI.SUM para obter a soma global.
- Compare com N(N+1)/2.

Exercício 3: maior valor global

Criar um vetor aleatório grande e encontrar o maior valor usando MPI.MAX.

- O processo 0 cria um vetor com 1 milhão de números aleatórios.
- O vetor é dividido entre os processos.
- Cada processo calcula seu máximo local.
- Use reduce com MPI.MAX.
- Compare o resultado com numpy.max().

```
maior_local = np.max(bloco)

maior_global = comm.reduce(
    maior_local,
    op=MPI.MAX,
    root=0
)
```

scatter

reduce + MAX

Exercício 4: média global

A média pode ser calculada com duas reduções: soma total e quantidade total.

Cada processo calcula:

```
soma_local = np.sum(bloco)
quantidade_local = len(bloco)
```

O processo 0 combina:

```
soma_global = reduce(SUM)
quantidade_global = reduce(SUM)

media = soma_global / quantidade_global
```

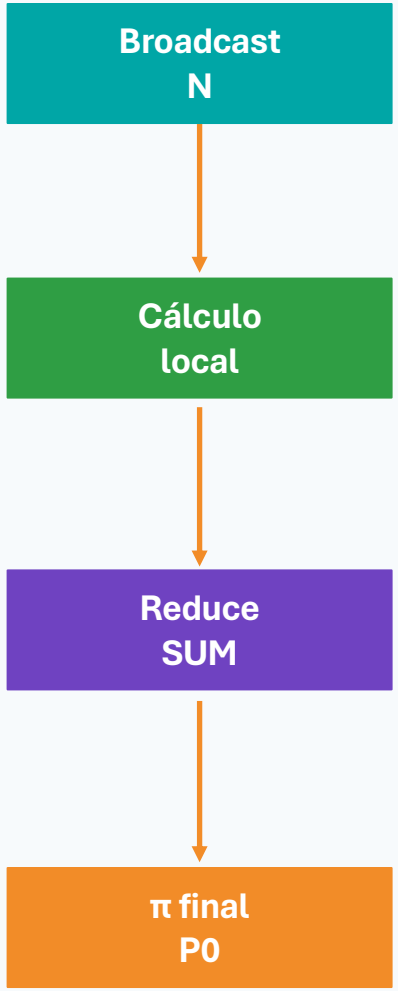
Gather não é necessário se queremos apenas a média final.

Desafio: cálculo paralelo de π

Aplicar o padrão completo em uma integração numérica.

$$\pi \approx \sum 4/(1+x_i^2) \cdot \Delta x$$

- Broadcast: distribuir N para todos os processos.
- Processamento paralelo: cada processo calcula parte da soma.
- Reduce: somar as contribuições locais.
- Processo 0: calcular o valor final e o erro.



Erros comuns em operações coletivas

Esses problemas são frequentes nas primeiras implementações com MPI.

- Nem todos os processos chamam a operação coletiva.
- A lista usada no scatter tem tamanho diferente do número de processos.
- Todos os processos imprimem o resultado final, gerando saída confusa.
- Usar gather quando reduce resolveria melhor o problema.
- Dividir dados de forma desbalanceada e deixar processos ociosos.

Boa prática: somente o rank 0 imprime tabelas, gráficos e resultados finais.

Observação: funções minúsculas e maiúsculas

Nesta aula usaremos a forma mais simples para destacar o conceito.

Objetos Python

```
comm.bcast()
comm.scatter()
comm.gather()
comm.reduce()
```

Buffers / NumPy

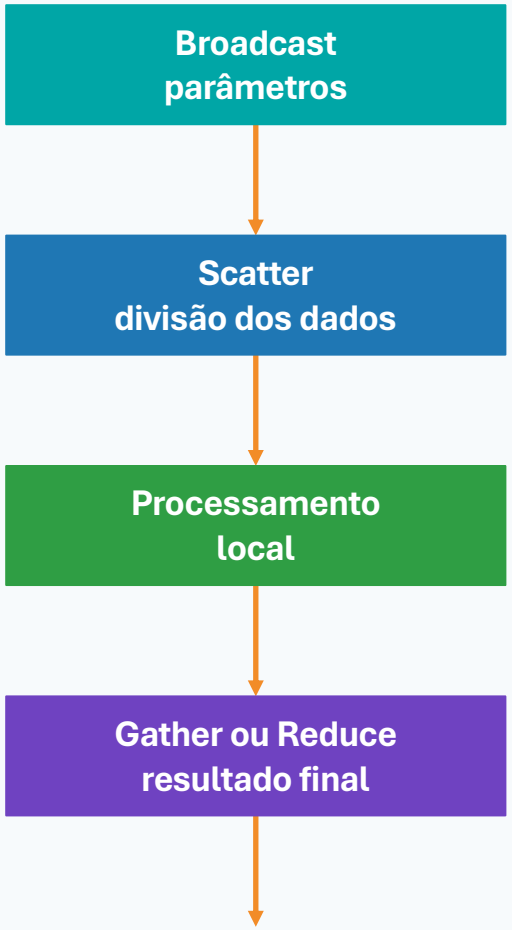
```
comm.Bcast()
comm.Scatter()
comm.Gather()
comm.Reduce()
```

Primeiro: entender o fluxo dos dados. Depois: otimizar comunicação com arrays NumPy.

Síntese da aula

O padrão abaixo será reutilizado nos desafios da disciplina.

Pergunta final:
qual operação coletiva resolve cada etapa do seu algoritmo?



Próximo passo:
aplicar esse padrão nos desafios de desempenho.